

What “tokens saved” means

Definition, guarantees and evidence behind the TokenSaver meter and the public counter

Revision 1.1 reconciles the Codex source and arithmetic review: meter and actual-token units, pilot-based experiment sizing, and narrower historical and implementation claims. Net task savings and preserved quality remain unmeasured.

AT A GLANCE

The number	An estimate of the input tokens that TokenSaver removed from requests before they reached the model provider: removed request-body bytes divided by four, counted on every request that would have carried the removed text.
Proved	An accepted rewrite never makes a request larger (Proposition 1). No lossy compression can preserve the answer to every possible task (Proposition 3).
Measured	On one developer workstation, from 4 September to 1 October 2026, accepted rewrites removed 3.45% of the request bytes Codex CLI submitted, 0.20% of Claude Code’s and 0.18% of Grok CLI’s.
Not measured	Avoided billable tokens or money relative to uncompressed tasks, net complete-task cost, and preserved task quality. Provider-reported usage was observed for the captured requests.
How to cite	“ <i>N</i> estimated input tokens not sent to model providers (removed bytes ÷ 4, counted per request).” Not “ <i>N</i> tokens saved on your bill”. Section 9 lists supported wording.

1 The number and how it is computed

TokenSaver is a proxy built into VibeRails. A coding CLI such as Claude Code, Codex CLI or Grok CLI sends its model requests to TokenSaver, which shortens selected tool output and forwards the request to the provider. Responses are passed back byte for byte.

1.1 The meter on each machine

For every recorded request j , TokenSaver knows the size B_j , in bytes, of the request body the CLI submitted, and the size A_j of the body it forwarded. The meter keeps the running sum

$$D = \sum_j (B_j - A_j), \quad T_{\text{est}} = \lfloor D / 4 \rfloor. \tag{1}$$

T_{est} is the number shown as tokens saved. The divisor 4 is a rule of thumb, about four bytes per token, applied when totals are read. No tokenizer is involved, and the result is not a provider’s token count.

A request is recorded only when all of the following hold:

- it is a POST with JSON Content-Type and a detectable body to a model inference path (/v1/messages, /responses or /chat/completions);
- TokenSaver is enabled for that CLI and is not paused;
- the body is at most 10 MiB (larger bodies are forwarded untouched and not recorded);
- the provider's response began with a 2xx status.

Requests in which nothing was rewritten are recorded with zero savings. Bodies are measured as received; TokenSaver does not decode HTTP content encodings.

1.2 Each re-send is counted

Coding CLIs can resend tool results as conversation history. With fixed input, command metadata, configuration and encoding, a carried result is rewritten consistently on qualifying requests. Compaction, server-managed history and changed settings can change which text is carried or rewritten. T_{est} counts byte/4 estimates of input tokens not transmitted, summed over qualifying requests. It is not a count of distinct content removed. On two sample days in August the meter's tally was 12.6 times (Claude Code) and 39.8 times (Codex CLI) the number of distinct characters removed. Those ratios mix bytes with characters and predate a 30 August change, so they illustrate the effect rather than measure a replay count.

1.3 The public counter

While an active VibeRails root backend is open, enabled publishing attempts to report its lifetime T_{est} using a configurable interval (15 minutes by default), when a usable API key and endpoint are available. For each pair of API key and computer name the server keeps the largest total that pair has reported, and the public endpoint sums those values with a 15-minute cache per server process. This describes inspected source, not an independent audit of deployed totals. It follows that:

- reports cannot lower a stored maximum for a key/computer pair, so a local database reset alone does not reduce that pair's reported total;
- each report is a lifetime total, so a computer that later reports under a second API key or a new computer name contributes its whole history again;
- the counter is a sum of per-machine estimates computed as in (1). The workload evidence in this note comes from one machine, and the counter's total has not been independently audited.

2 What TokenSaver changes

TokenSaver rewrites only tool results, the text returned by commands an agent has run, and only for tools on an allow-list: shell commands for all three CLIs, including Codex code-mode execution and Grok sub-agent output. System prompts, text the user typed, tool-call arguments, tool definitions and model responses are never rewritten. In the Anthropic Messages API tool results travel inside user-role messages; only the tool-result blocks are rewritten, and any text a client places inside them is rewritten with them. The dedicated file-reading and search tools are excluded by default.

Table 1. Stages that run by default, in order.

Stage	Kind	Effect
Line-ending and whitespace cleanup	Lossless	Converts CRLF line endings to LF, strips trailing spaces and tabs, trims blank lines at the start and end, and shortens runs of three or more blank lines to two.

Stage	Kind	Effect
Grouping of <code>git status</code> , <code>grep</code> and <code>find</code> output	Reshaping	Rearranges recognised output into a more compact layout.
Passing-test collapse	Lossy	Replaces runs of passing-test lines from recognised test runners with <code>[... N passed ...]</code> ; failures, skips and summaries stay.
Repeated-line collapse	Lossy	Writes three or more identical consecutive lines once, followed by <code>[xN]</code> .
Long-output truncation	Lossy	Keeps the first 150 and the last 50 lines and replaces the middle with <code>[... N lines elided ...]</code> , but only when at least 10 lines and 4,096 UTF-16 code units would be removed.
File-read budget	(limit)	For recognised file reads, truncation keeps 1,200 and 200 lines instead, provided the post-dedupe output is at most 262,144 UTF-16 code units; larger reads fall back to 150 and 50.

Further limits decide what is removed:

- Any output that contains an escape (ESC) or bell character, or a carriage return outside a Windows line ending, is left whole by the lossy stages. ANSI colour stripping is off by default, so coloured or progress-bar output is not truncated.
- Historical sources report a Claude Code Bash cap near 30,000 characters, with 29,934 observed in the tiered window. This is not a universal current-client guarantee or a 30 KB byte limit.
- File reads are recognised from the command (`cat`, `head`, `tail`, `sed`, `Get-Content` and similar). Reads through `awk` are not recognised and get the ordinary budget, and diffs are truncated deliberately.
- A candidate replacement is used only if it is smaller than the original in its encoded form (Section 3). If anything fails, the original request is forwarded unchanged.

3 Guarantee: a rewrite never enlarges a request

Proposition 1 (byte non-growth). Write a request body as the byte string

$$B = v_0 s_1 v_1 s_2 \cdots s_n v_n,$$

where $s_1, \dots, s_n$ are the encoded JSON string tokens selected for rewriting, in order and disjoint, and $v_0, \dots, v_n$ are the bytes between them. For each i let c_i be the encoded candidate produced by the pipeline, and set $a_i = c_i$ if $|c_i| < |s_i|$ and $a_i = s_i$ otherwise, where $|\cdot|$ is length in bytes. The forwarded body is $R(B) = v_0 a_1 v_1 \cdots a_n v_n$. Then

$$|R(B)| = \sum_{k=0}^n |v_k| + \sum_{i=1}^n |a_i| \leq \sum_{k=0}^n |v_k| + \sum_{i=1}^n |s_i| = |B|, \quad (2)$$

with equality if and only if no candidate is accepted, and every byte outside the accepted tokens is unchanged.

Proof. Concatenation adds lengths, which gives the two equalities. By construction $|a_i| \leq |s_i|$ for every i , with strict inequality exactly when c_i is accepted. Summing over i gives the inequality, which is strict as soon as one candidate is accepted. The v_k are copied unchanged by construction, and a rejected token is replaced by itself. ■

Corollary. Every recorded request contributes $B_j - A_j \geq 0$, so the meter can only increase.

How the implementation matches the model. Each of the three provider rewriters (Anthropic Messages, OpenAI Responses and Chat Completions) copies the bytes between selected tokens verbatim, re-encodes a candidate, and accepts it only when its encoded byte count is strictly smaller than the original token's, which preserves client escaping outside accepted spans; accepted spans are re-encoded. Any failure forwards the original body. This correspondence was established by reading the source at commit 336373c0; it is not machine-verified. Regression tests assert whole-body byte identity outside rewritten tokens for the Anthropic rewriter.

What Proposition 1 does not say.

- *Tokens*: tokenizers are not monotone in byte length, so a shorter string can in principle produce more tokens.
- *Cost*: a smaller request can cost more if its bytes differ from a prefix the provider has already cached (Section 6).
- *Meaning*: it says nothing about whether the removed text mattered (Section 7).

4 Design invariants and tests

With a fixed configuration, every stage is designed to be deterministic (a function of the text and tool/command metadata), idempotent (applying it again changes nothing) and never-growing. Reproducing earlier rewritten bytes helps preserve cached prefixes; it does not guarantee a hit, which also depends on expiry, cache boundaries and provider behavior.

- **Tests.** 475 automated tests in ten TokenSaver test classes passed on 5 October 2026 (.NET SDK 10.0.401). They cover cleanup, condensation, golden fixtures, command shaping, file-read budgets, line endings and the three provider rewriters, including escaping edge cases (emoji and lone surrogates) for the Anthropic rewriter. Limits: adversarial escaping coverage is concentrated in Anthropic; Codex also checks whole-body no-op equality and byte shrinkage, with narrower changed-body checks; composition is tested on a two-stage fixture rather than the shipped pipeline; there are no randomised (property-based) tests; and the meter's own accounting tests were not part of this run.
- **Replay.** A historical study ran 8,177 distinct recorded tool outputs through the pipeline under four configurations (32,708 cases) and found no growth, non-determinism or non-idempotence. Its inputs were the texts that had actually been forwarded, so it tests the stability of already-compressed text rather than the raw-to-compressed step. Re-running the then-current defaults occasionally changed a recorded text, mostly in whitespace; pauses, older builds, command metadata and guard decisions are possible explanations, not resolved attributions.
- **Live traffic.** Among 64 investigated Claude Code Window-B cache misses, each writing more than 50,000 cache tokens within five minutes of the previous request in the same conversation, none diverged at a differently compressed tool result.
- **A known exception.** Settings and the pause switch are read on every request, and the whole history is recompressed each time. Pausing TokenSaver, or changing its settings, can alter previously sent bytes. The product documentation warns of two cache-transition opportunities per pause, but actual misses depend on changed bytes, cached prefixes and provider behavior. Paused requests are not recorded by the meter.

5 Measured results

Table 2. Request-body bytes removed on one developer workstation, activity from 4 September to the first snapshot of 1 October 2026. Decimal units (1 GB = 10⁹ bytes).

CLI	Requests	Submitted	Removed	Share	Estimated tokens (removed ÷ 4)
Codex CLI	16,227	9.93 GB	343 MB	3.45%	85.8 million
Claude Code	16,238	7.71 GB	15.4 MB	0.20%	3.9 million
Grok CLI	1,212	673 MB	1.2 MB	0.18%	0.3 million

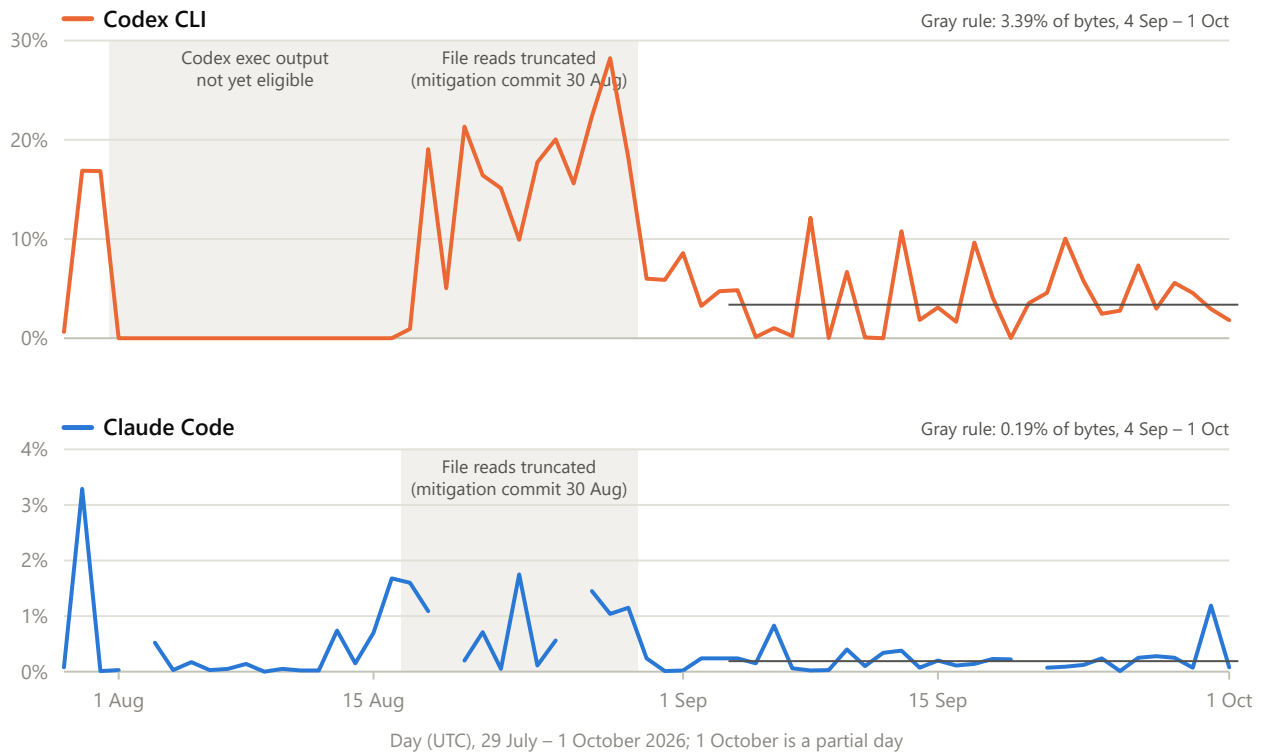


Figure 1. Daily share of submitted request bytes removed, per CLI. The two panels use different vertical scales. Shaded periods are explained in Table 3. The horizontal rules mark the byte-weighted share for 4 September – 1 October in a later reading of the same meter. Data: `daily-series.csv` in the evidence package.

Table 3. Byte-weighted share removed by period (sums of the rounded daily rows plotted in Figure 1).

Period	Codex CLI	Claude Code	What changed
29 Jul – 16 Aug	1.02%	0.20%	Codex code-mode exec output was not yet eligible; from 1 August no Codex request was rewritten.
17 – 29 Aug	14.93%	1.11%	exec output allow-listed; file reads were truncated like logs.
30 Aug – 3 Sep	5.64%	0.20%	File reads given the larger budget.
4 Sep – 1 Oct	3.39%	0.19%	No pipeline changes shipped.

Reading the results.

- The share depends on the workload. Codex days since 4 September ranged from 0.00% to 12.14% (median 3.04%), and five days account for 54% of the bytes Codex removed. A recount of 4–8 September

from the request archive gives 1.41% for Codex and 0.23% for Claude Code. That archive analysis selected HTTP-200 model requests and excluded model-list/count-token endpoints. It uses a different population definition from the meter; the size of the selection effect is unmeasured.

- About half of the Claude Code requests in that recount matched its known large-system/no-tools classifier family. If that family contributed zero removed bytes, subtracting its system-character lower bound on serialized bytes gives a conservative non-family reduction of at least 0.295%. The exact family byte subtotal and zero-removal condition were not independently established.
- From 17 to 29 August, file reads were being truncated like logs, which was treated as a defect and changed on 30 August. Codex's share fell from 14.93% in that period to 5.64% in the five days after the change and 3.39% from 4 September. Lifetime totals, including the public counter, still contain the bytes removed in August.
- These shares compare each forwarded request with the request the CLI submitted. They are not the reduction relative to running without TokenSaver, because the submitted requests already include any follow-up reads that earlier compression may have caused (Section 7).

6 From estimated tokens to cost

6.1 Bytes are not tokens

Providers bill tokens produced by model-specific tokenizers, not bytes. Dividing by four is an estimate, and the JSON escaping of tool output inflates bytes relative to tokens. For orientation only: in the 4–8 September recount, submitted Claude Code request bytes divided by forwarded-request input usage total about 3.0 bytes per reported input token. That whole-body average includes framing and tool definitions and is not a calibration for the removed text. Exact differences require counting tokens for the original and the forwarded body of each request with the provider's tokenizer (Section 10).

6.2 Re-sent input is mostly billed at the cache-read price

Providers can discount input that repeats a cached prefix. The historical models use Anthropic-style multipliers of 1.25 for a five-minute write, 2 for a one-hour write and 0.1 for a read; the no-write-premium curve is an OpenAI-style example. These are illustrative parameters, not independently verified current prices for every model. In the 4–8 September recount, 96.6% of Claude Code's provider-reported input tokens were cache reads. Pin the actual model, pricing date and billing agreement before converting usage to money.

Proposition 2 (avoided cost of a removed block). Consider a block of d tokens that would appear in m requests of a conversation: written to the cache at multiplier w in the first and read at multiplier α in each of the other $m - 1$, with nothing else changing. Removing it avoids the input cost

$$G = d [w + \alpha(m - 1)] \quad (3)$$

in units of the base input price. Dividing by the $d \cdot m$ actual token appearances gives avoided cost per actual token appearance:

$$v(m) = \frac{w + \alpha(m - 1)}{m}, \quad (4)$$

which falls as m grows whenever $w > \alpha$, and approaches α .

Proof. The block is paid for once at w , when the first request carrying it writes it to the cache, and $m - 1$ times at α , when later requests read it. With nothing else changing, removing it avoids exactly G . There are $d \cdot m$ actual token appearances, so dividing by that quantity gives (4). The product meter instead divides removed bytes by four; equation (4) is not its monetary conversion. Its derivative is $\frac{dv}{dm} = \frac{\alpha - w}{m^2}$, which is negative when $w > \alpha$, and $v(m) \rightarrow \alpha$ as $m \rightarrow \infty$. ■

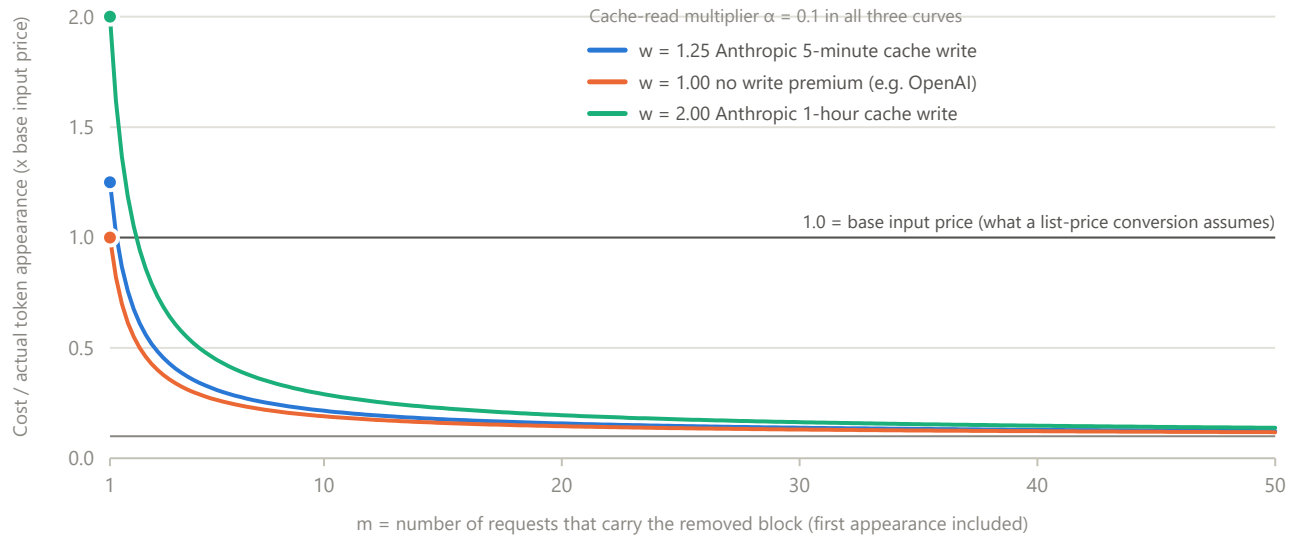


Figure 2. The value of one actual token appearance, relative to the base input price, under Proposition 2 with $\alpha = 0.1$. Each curve starts at its write multiplier w (dot) and approaches the cache-read price, 0.1 (thin rule near the bottom). Values at selected m are in Table 4.

Table 4. Avoided cost per actual token appearance, $v(m)$, with $\alpha = 0.1$.

m	$w = 1.25$ (Anthropic, 5-minute write)	$w = 1$ (no write premium)	$w = 2$ (Anthropic, 1-hour write)
1	1.250	1.000	2.000
5	0.330	0.280	0.480
10	0.215	0.190	0.290
20	0.158	0.145	0.195
30	0.138	0.130	0.163
50	0.123	0.118	0.138

Consequence. With the assumed write/read prices and unchanged cache behavior, repeated actual token appearances carry a lower average price than uncached input. This does not price the byte/4 meter. If each appearance removes b encoded bytes and d actual input tokens, avoided cost per meter-estimated token is approximately $(4d/b)$ times equation (4), ignoring aggregate integer rounding. Here d must be the complete-request token delta attributable to the block; an isolated substring token count need not equal that delta. The full-request token delta, byte delta and cache categories must be measured before converting the meter to money. Expiry, misses and changed continuations require separate accounting.

6.3 Subscriptions

On flat-rate subscription plans the relevant quantity is consumption of usage limits rather than billed tokens. This note does not establish how cached input counts toward those limits.

7 What compression can cost

7.1 Follow-up reads

Truncation leaves a marker so the agent can ask for the missing part. A heuristic classifier found a related call in the next turn after 19 of 35 Claude Code truncations and 146 of 448 classified Codex CLI truncations, in conversations first seen between 30 August and 1 October. No comparison group of untruncated outputs was analysed, so these rates do not show that truncation caused the calls. The classifier can count ordinary next steps, such as opening a file after reading its diff; their prevalence is not measured against a control group.

7.2 A model of the net effect

Let G be the avoided cost of the lossy removals (Proposition 2), q the probability that a removal causes an extra recovery, L the expected cost of that recovery, r the probability that a removal silently degrades the result, F the cost of that degradation, and K any other effect, which may be positive (for example, cache breaks) or negative (for example, later or fewer context compactions). The probabilities q and r are dimensionless. G , L , F and K use a common monetary unit; F requires an explicit valuation of quality loss. Specify one intervention and continuation, include all recovery within it and avoid overlapping costs. Without that valuation, report monetary cost and quality separately. The expected net saving is

$$E[N] = G - qL - rF - K, \quad (5)$$

which is positive exactly when

$$q < q^* = \frac{G - K - rF}{L} \quad (L > 0). \quad (6)$$

Equation (5) is accounting, not a measurement: q , L , r and F have not been measured. A historical sensitivity model, which charged a whole extra turn for every related call and used provider-wide prices on traffic from 29 July to 3 September, most of it before the 30 August change, found truncation net negative for Claude Code (break-even near a 19% re-run rate, against related-call rates of 40–75% under two classifiers) and net positive for Codex CLI. A structural reason is plausible: Claude Code's own 30,000-character cap leaves little for truncation to remove, while each extra turn re-reads a long cached context.

7.3 What lossy compression can preserve

Proposition 3 (no lossy compression preserves every task). For a nonempty task domain X , let $\varphi: X \rightarrow Z$ map full tool outputs to compressed outputs, and let $g: X \rightarrow Y$ give the correct answer to some task. A decoder $h: Z \rightarrow Y$ with $h(\varphi(x)) = g(x)$ for every $x \in X$ exists if and only if

$$\forall x, y \in X: \varphi(x) = \varphi(y) \Rightarrow g(x) = g(y). \quad (7)$$

Proof. If h exists and $\varphi(x) = \varphi(y)$, then $g(x) = h(\varphi(x)) = h(\varphi(y)) = g(y)$. Conversely, if (7) holds, define $h(z)$ as the common value of g on the outputs that φ maps to z , and arbitrarily where there are none. Condition (7)

makes this well defined, and $h(\varphi(x)) = g(x)$ by construction. ■

Corollary. Truncation is not injective on an unrestricted output domain. Two outputs with the same retained lines and omitted-line count can differ in their removed middle lines yet compress identically. For a task that distinguishes those outputs, (7) fails: no decoder can answer correctly for both from the compressed text alone. A restricted task domain can satisfy (7); deployment quality still needs evaluation on task outcomes. An agent may attempt recovery with a narrower command, incurring the recovery cost L of Section 7.2, without a guarantee of success.

8 What has not been established

1. **Net cost.** Whether the total cost of completing a task falls once follow-up reads, cache effects and retries are included. Not measured for any CLI.
2. **Quality.** Whether agents complete tasks equally well with compressed output. Not measured.
3. **Generality.** All workload results come from one developer’s workstation and task mix, in which the analysts’ own agent sessions also appear.
4. **Causality of follow-ups.** No base rate exists for the follow-up classification (Section 7.1).
5. **The public counter.** Computed as described in Section 1.3. Its server-side total has not been audited, and lifetime totals include the August period in which file reads were truncated.

9 How to state the claim

Table 5. Wording this note supports, and wording it does not.

Supported	Not supported
“TokenSaver has kept an estimated N input tokens from being sent to model providers.” Footnote: “estimated as removed request bytes $\div 4$, counted on every request that would have carried them.”	“Saved N tokens on your bill”, “saved \$ X ”, or any conversion of N to money at list prices.
“TokenSaver never makes a request larger: a rewrite is used only when it is smaller, byte for byte.”	“Never changes what the model needs to know”, or “no effect on quality”.
“On our own workstation, from 4 September to 1 October 2026, TokenSaver removed 3.45% of Codex CLI request bytes, 0.20% of Claude Code’s and 0.18% of Grok CLI’s.”	A general rate applied to other workloads (“cuts tokens by 3.45%”), or the 15–28% days of late August.
“TokenSaver trims tool output only. It never rewrites your prompts or the model’s replies, and it forwards the original request if anything goes wrong.”	“Lowers the cost of completing a task.”

10 How the open questions can be answered

1. **Gross token differences.** Count tokens for the original and forwarded requests using a model-compatible full-request tokenizer or provider counting endpoint. Check the counting method’s documented limits, pricing and agreement with provider-reported usage; this calibrates the $\div 4$ estimate without running a complete task.

2. **A base rate for follow-ups.** Apply the same classifier to untruncated outputs, to outputs just below the truncation threshold, and to Codex traffic from 1–16 August, when no Codex request was rewritten.
3. **A fork experiment.** Branch recorded conversations at each truncation and run the next turns with the full and with the truncated output from the same cached prefix, measuring follow-up calls, cost and the answer.
4. **Claude Code without truncation.** The underlying research recommends a before-and-after comparison with truncation switched off. Such a comparison is exploratory: workload, model and cache changes can confound it; causal attribution needs matched or randomized controls.
5. **Complete-task trials** for quality and cost noninferiority. A pilot must determine the paired task-cost variance, detectable effect and noninferiority margins. For an assumed paired log-cost SD of 0.3, a 0.2% effect needs about 176,000 independent pairs at two-sided 5% significance and 80% power. Gross byte percentages are not task-cost effect sizes; neither variance nor impossibility is established here.

Appendix A Notation

Symbol	Meaning
B_j, A_j	Bytes of request j as submitted by the CLI and as forwarded
D, T_{est}	Total bytes removed; estimated tokens, $\lfloor D / 4 \rfloor$
s_p, c_p, a_p, v_k	Selected string tokens, candidates, accepted outputs, untouched bytes (Proposition 1)
d, m, w, α	Block size in tokens, number of requests carrying it, cache-write and cache-read multipliers (Proposition 2)
$G, v(m)$	Avoided cost of a block; avoided cost per actual token appearance
q, L, r, F, K	Recovery probability and cost; silent-degradation probability and cost; other effects (Section 7.2)
φ, g, h	Compression map, task answer, decoder (Proposition 3)

Appendix B Sources and provenance

- **Product source.** VibeRails repository, TokenSaver library, read at commit 336373c0 (unchanged at the time of writing). Acceptance gate: AnthropicMessagesRewriter.cs:188, CodexResponsesRewriter.cs:209, ChatCompletionsRewriter.cs:176. Meter: ITokenSavingsStore.cs:26 and LlmProxyRelay.cs:140–158. Stage catalogue: CompressionCatalog.cs.
- **Public counter.** viberails.ai server source: TokenSavingsController (one value per API key and computer name, increases only) and PublicTokenSavingsController (sum of all values, cached for 15 minutes).
- **Measurements.** TokenSaver research draft 0.2 and its evidence package (aggregates and the daily series), 5 October 2026, drawn from an internal research archive at commit cf3253b. Raw captured traffic is private and is not published.
- **Review.** A separate review of draft 0.2 (findings R-01 to R-20, 5 October 2026), whose proposed corrections were independently checked by Codex; accepted findings and disagreements are documented in the 0.3 review and technical-note supplement.
- **External.** The original author cites Anthropic developer documentation, “Prompt caching” and “Token counting”, accessed 5 October 2026. This review did not independently retrieve those pages; current

model-specific prices and counting guarantees remain unverified.

- **Method.** The analyses were carried out by AI coding agents working under the developer's direction, using source inspection and arithmetic on saved aggregates. No new traffic was collected for this note.

Appendix C Numerical checks

- Table 2 shares: $343 / 9,930 = 3.454\%$; $15.4 / 7,710 = 0.1997\%$; $1.2 / 673 = 0.178\%$. Estimated tokens: $343 \text{ MB} \div 4 = 85.75 \text{ million}$; $15.4 \text{ MB} \div 4 = 3.85 \text{ million}$; $1.2 \text{ MB} \div 4 = 0.30 \text{ million}$.
- Section 5 recount (4–8 September): $21,603,929 / 1,532,592,995 \text{ bytes} = 1.410\%$ (Codex); $3,889,891 / 1,698,925,099 = 0.229\%$ (Claude Code). The classifier calls' system prompts alone total 381,468,024 characters, so, conditional on zero removal in that family, subtracting that byte lower bound gives at least $3,889,891 / 1,317,457,075 = 0.295\%$; this is not a matched family-byte exclusion.
- Section 6: $539,740,748 \text{ cache-read tokens of } 558,868,027 \text{ input tokens} = 96.6\%$; $1,698,925,099 \text{ bytes} / 558,868,027 \text{ tokens} = 3.04 \text{ bytes per token}$.
- Section 7: $19 / 35 = 54\%$; $146 / 448 = 33\%$.
- Table 4: equation (4) with $\alpha = 0.1$, for example $v(10) = (1.25 + 0.9) / 10 = 0.215$.